

RideFlow Launch Readiness and Account Administration Guide

Repository: <https://github.com/raiven427/ride-flow>

Author: Manus AI

Purpose: Help the owner verify the protected application flows, understand the next GPS and dispatch build, retain operational activity safely, select payment methods, and change administrator credentials without exposing secrets.

Important: The password previously shared in chat should be treated as exposed. Do not reuse it for production. Rotate it through your identity provider or hosting secret manager before launch.

1. What was verified in the preview

The public preview was checked without a signed-in account. The customer overview loaded with truthful new-installation values: KSh 0 saved, zero favorite rides, zero safety check-ins, and a zero fare placeholder. The customer booking screen rendered the route preview, driver choices, preferences, safety option, transparent KSh fare breakdown, and the quote button.

Submitting the quote while unauthenticated displayed the intended protected-flow fallback: RideFlow showed a message asking the user to sign in to save a live quote and kept the current demo estimate visible. The driver mode showed zero earnings, zero take-home, zero acceptance, no pending requests, and a zero weekly fee breakdown. The profile page rendered the upload control and settings.

A fully authenticated test could not be completed from the sandbox session because the OAuth login requires the account owner to enter credentials in the open browser. The owner must complete that step manually; passwords should not be sent through chat or embedded in source code.

2. How to complete the signed-in browser test

First open the private RideFlow preview and use the **Sign up** or sign-in entry point. Take over the browser and enter the credentials directly into the identity provider. Complete any verification challenge. Return to RideFlow after the callback.

Test	Expected result	Evidence to record
Customer sign-in	The session returns to RideFlow and protected calls stop showing the sign-in fallback	Account email and timestamp, without recording the password
Fare quote	Review ride calls the server-side fare procedure and displays the returned KSh calculation	Quote total and server response status
Profile upload	Selecting a JPG, PNG, or WEBP stores metadata and shows a success toast	File purpose, size, and success state; never retain private document bytes in screenshots
Driver onboarding	Driver documents require an authenticated account and remain pending review	Upload status and review state
Admin operations	Only the admin sees the operations navigation and can load online users/activity	Admin role, account count, heartbeat state, and activity count
Ordinary-user authorization	A non-admin cannot call the admin operations procedure directly	Expected forbidden response
Logout	Session is cleared and protected actions return to the login flow	Logout result and follow-up protected request

If a test fails, collect the browser console message, the server log timestamp, the request path, and the database error. Do not paste access tokens, cookies, passwords, payment secrets, or identity-document contents into an issue report.

3. Changing the administrator email and credential

RideFlow has two separate concepts: the **application admin role** and the **identity-provider login**. The admin role is transferred inside RideFlow's Admin Control Room. The password is not stored in RideFlow's database when using OIDC; it belongs to the identity provider.

To change the admin email, sign in as the current admin, open **Profile**, open the Admin Control Room, and use **Transfer admin ownership**. The destination account must sign in to RideFlow

first. Enter the new email, confirm the transfer, then sign out and sign in with the new account. Test that the old account no longer sees Admin operations.

To change the password, open the identity provider's account-security page, choose **Change password** or **Reset password**, create a unique password, and enable multi-factor authentication if available. Then update any deployment secret only if the provider specifically requires a client secret rotation. Do not place a password in `.env`, `client/src`, `server/`, GitHub, screenshots, or PDFs.

Credential or setting	Where to change it	What to test afterward
RideFlow admin ownership	Profile → Admin Control Room → Transfer admin ownership	New account has admin operations; old account does not
Identity-provider password	Identity provider security settings	New login works; old password fails
OIDC client secret	Hosting secret manager and provider console	Login callback succeeds in a private browser
Database password	Database provider and <code>DATABASE_URL</code> secret	<code>/healthz</code> , migrations, and authenticated database calls
Stripe keys	Stripe dashboard and deployment secrets	Sandbox payment intent and webhook verification
Daraja credentials	Safaricom developer portal and deployment secrets	Sandbox STK request and callback verification

4. Payment-method selector

The Profile page now includes a payment-method selector above Profile photo. The available choices are **Cash**, **M-Pesa**, **PayPal**, and **Cards**. The selector is currently a preference UI: it records the selected option in the page state and explains that provider details are collected only after the corresponding provider is configured.

Before accepting real payments, connect each method to a server-side payment flow. Never trust an amount sent from the browser. The server must calculate the fare, create the provider transaction, verify the provider callback or webhook, and write a reconciled ledger record. Stripe recommends verifying webhook signatures using the raw request body [1]. Safaricom's Daraja platform provides the official developer and API documentation for M-Pesa integrations [2].

5. GPS and dispatch: what must be built next

The current RideFlow project has a route-preview UI, but it is not yet a production dispatch or live-tracking system. A safe implementation should introduce explicit trip states: `requested`, `matching`, `driver_assigned`, `driver_arriving`, `driver_waiting`, `in_progress`, `completed`, `cancelled`, and `disputed`.

The dispatch service should select eligible drivers using verified status, current availability, vehicle capability, women-only eligibility where legally and operationally supported, distance or ETA, acceptance constraints, and current trip load. Matching should be server-side and idempotent. A retry must not assign the same ride twice.

GPS updates should be sent only while the relevant trip is active and only to authorized viewers. Store the latest location separately from the audit trail. The latest location can be overwritten; operational events such as assignment, pickup, cancellation, and completion should remain append-only. Encrypt transport with HTTPS/WSS and minimize the precision and retention of historical location data.

Two viable deployment approaches

Approach	Tradeoffs	Cost	Setup complexity
Managed realtime service with WebSocket or publish/subscribe support	Faster presence and location delivery, but adds vendor configuration, data-processing review, and monthly usage cost	Provider-dependent	Medium
Self-hosted WebSocket service beside RideFlow	Maximum control and portable protocol, but requires persistent hosting, monitoring, reconnect handling, and scaling work	Server and operations cost	High

For an early pilot, use a managed realtime provider or a small persistent WebSocket service. Do not use a low-frequency scheduled task for sub-minute driver location updates. A realtime connection or persistent process is the correct design for event-driven tracking; scheduled tasks are not suitable for minute-level polling [3].

6. Activity retention policy

RideFlow's activity feed records sign-ins, presence events, fare quotes, uploads, notification changes, and admin transfers. The current dashboard shows recent activity, but production should define retention before launch.

A practical starting policy is to keep detailed operational events for 90 days, keep security-sensitive admin events for 12 months or the period required by local policy, and delete or aggregate older records. Store only the minimum metadata needed for support and security. Do not put payment credentials, passwords, access tokens, raw identity documents, or precise historical GPS traces in the generic activity table.

Retention should run as deterministic server-side maintenance, not as an AI task. On a managed host, run a daily database job or host-native cron that deletes records older than the approved window in small batches. On an always-on host, use a worker or scheduled process with metrics and failure alerts. Keep a dry-run mode and test the deletion query against a backup or staging database before enabling it.

A safe policy change process is: write the retention decision in the operations handbook; add an index on the timestamp column; run a bounded delete in staging; verify that recent events remain; monitor duration and affected-row counts; then enable production cleanup. Do not run an unbounded `DELETE` during peak traffic.

7. Launch sequence

Complete identity-provider login and the signed-in browser test first. Then connect a staging database and verify quote creation, file metadata, admin authorization, and activity writes. Configure Stripe and Daraja sandbox credentials, test callbacks, and reconcile the resulting ledger entries. Build dispatch and location authorization before inviting real drivers. Add monitoring for API errors, payment callbacks, database connectivity, WebSocket connections, and retention-job failures.

A production launch should also include driver identity and vehicle verification, insurance and licensing review, customer support escalation, refund and dispute workflows, incident response, data-export/deletion procedures, and a review of Kenya-specific transport, privacy, payment, and employment requirements with qualified local advisers.

References

[1]: <https://docs.stripe.com/webhooks> Stripe, "Webhooks."

[2]: <https://developer.safaricom.co.ke/> Safaricom, "Daraja Developer Portal."

[3]: <https://ubuntu.com/server/docs> Ubuntu, “Server documentation,” for operational hosting and service administration concepts.